



# CSE 3313: Computer Architecture

**Sidratul Tanzila Tasmi**

Lecturer, Department of Computer Science and Engineering  
United International University

# MIPS machine language

| Name       | Format | Example |        |        |         |        |        | Comments                      |
|------------|--------|---------|--------|--------|---------|--------|--------|-------------------------------|
| add        | R      | 0       | 18     | 19     | 17      | 0      | 32     | add \$s1,\$s2,\$s3            |
| sub        | R      | 0       | 18     | 19     | 17      | 0      | 34     | sub \$s1,\$s2,\$s3            |
| lw         | I      | 35      | 18     | 17     | 100     |        |        | lw \$s1,100(\$s2)             |
| sw         | I      | 43      | 18     | 17     | 100     |        |        | sw \$s1,100(\$s2)             |
| and        | R      | 0       | 18     | 19     | 17      | 0      | 36     | and \$s1,\$s2,\$s3            |
| or         | R      | 0       | 18     | 19     | 17      | 0      | 37     | or \$s1,\$s2,\$s3             |
| nor        | R      | 0       | 18     | 19     | 17      | 0      | 39     | nor \$s1,\$s2,\$s3            |
| andi       | I      | 12      | 18     | 17     | 100     |        |        | andi \$s1,\$s2,100            |
| ori        | I      | 13      | 18     | 17     | 100     |        |        | ori \$s1,\$s2,100             |
| sll        | R      | 0       | 0      | 18     | 17      | 10     | 0      | sll \$s1,\$s2,10              |
| srl        | R      | 0       | 0      | 18     | 17      | 10     | 2      | srl \$s1,\$s2,10              |
| beq        | I      | 4       | 17     | 18     | 25      |        |        | beq \$s1,\$s2,100             |
| bne        | I      | 5       | 17     | 18     | 25      |        |        | bne \$s1,\$s2,100             |
| slt        | R      | 0       | 18     | 19     | 17      | 0      | 42     | slt \$s1,\$s2,\$s3            |
| j          | J      | 2       | 2500   |        |         |        |        | j 10000 (see Section 2.9)     |
| jr         | R      | 0       | 31     | 0      | 0       | 0      | 8      | jr \$ra                       |
| jal        | J      | 3       | 2500   |        |         |        |        | jal 10000 (see Section 2.9)   |
| Field size |        | 6 bits  | 5 bits | 5 bits | 5 bits  | 5 bits | 6 bits | All MIPS instructions 32 bits |
| R-format   | R      | op      | rs     | rt     | rd      | shamt  | funct  | Arithmetic instruction format |
| I-format   | I      | op      | rs     | rt     | address |        |        | Data transfer, branch format  |

# MIPS to Machine Language Translation



MIPS Fields:

3 types

- R-Type (add, sub, and, or, nor, sll, srl, slt, jr)
- I-Type (lw, sw, andi, ori, addi, beq, bne)
- J-Type (j, jal)

# How to Remember type of an instruction?

R-type (Register type)

Uses 3 registers: op rd, rs, rt

Example:

add \$t0, \$t1, \$t2

sub \$s1, \$s2, \$s3

and \$a0, \$a1, \$a2

**Pattern to remember: If it has 3 registers (destination + 2 sources), it's usually R-type.**



# How to Remember type of an instruction?

I-type (Immediate type)

Uses 2 registers + immediate (constant / address offset).

Format: op rt, rs, immediate

Examples:

`addi $t0, $t1, 5`

`lw $t0, 4($sp)`

**Pattern to remember: If it has a constant number (immediate) or memory offset, it's I-type.**

# How to Remember type of an instruction?



J-type (Jump type)

Only jump instructions (j, jal).

Has only an address (no registers).

Examples:

j LABEL

jal FUNCTION

Pattern to remember: **If it's a jump with only a label/target**, it's J-type.

# R Type



|        |        |        |        |        |        |
|--------|--------|--------|--------|--------|--------|
| op     | rs     | rt     | rd     | shamt  | funct  |
| 6 bits | 5 bits | 5 bits | 5 bits | 5 bits | 6 bits |

- *op*: Basic operation of the instruction, traditionally called the **opcode**.
- *rs*: The first register source operand.
- *rt*: The second register source operand.
- *rd*: The register destination operand. It gets the result of the operation.
- *shamt*: Shift amount. (Section 2.5 explains shift instructions and this term; it will not be used until then, and hence the field contains zero.)
- *funct*: Function. This field selects the specific variant of the operation in the *op* field and is sometimes called the *function code*.

# R Type



Assembly Instruction:

add \$s0, \$s1, \$s2



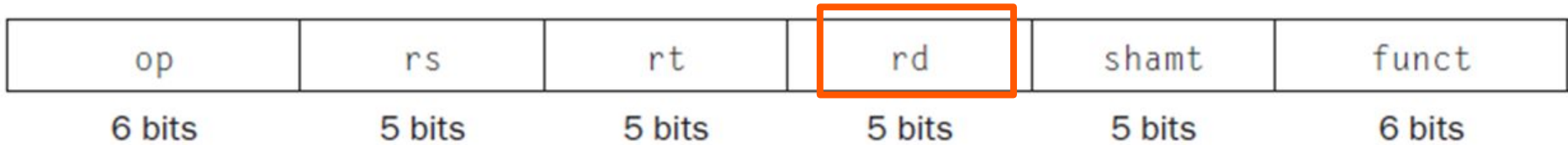


# R Type



Assembly Instruction:

add \$s0, \$s1, \$s2

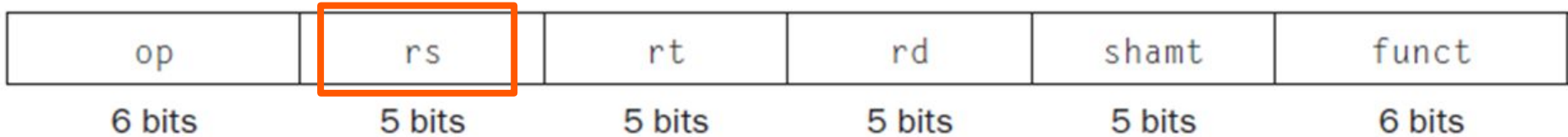


# R Type



Assembly Instruction:

add \$s0, \$s1 \$s2

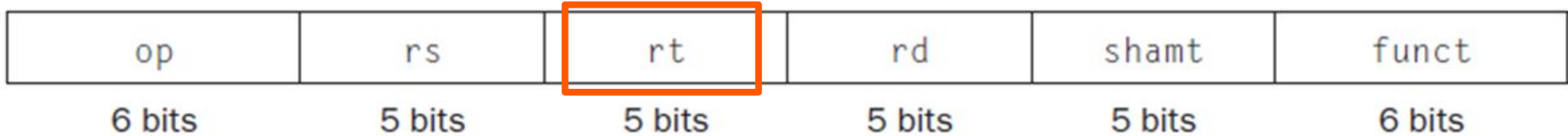


# R Type



Assembly Instruction:

add \$s0, \$s1, \$s2

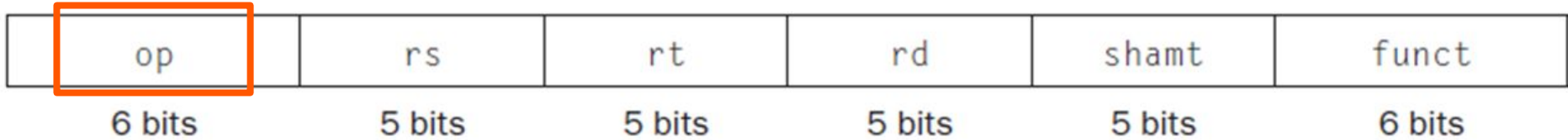


# R Type



Assembly Instruction:

`add $s0, $s1, $s2`



# MIPS Register Numbering



| Name      | Register number | Usage                                        |
|-----------|-----------------|----------------------------------------------|
| \$zero    | 0               | the constant value 0                         |
| \$v0-\$v1 | 2-3             | values for results and expression evaluation |
| \$a0-\$a3 | 4-7             | arguments                                    |
| \$t0-\$t7 | 8-15            | temporaries                                  |
| \$s0-\$s7 | 16-23           | saved                                        |

| Name      | Register number | Usage            |
|-----------|-----------------|------------------|
| \$t8-\$t9 | 24-25           | more temporaries |
| \$gp      | 28              | global pointer   |
| \$sp      | 29              | stack pointer    |
| \$fp      | 30              | frame pointer    |
| \$ra      | 31              | return address   |

# Where to get Opcode and Function Code

MIPS machine language

| Name       | Format | Example |        |        |         |        |        | Comments                      |
|------------|--------|---------|--------|--------|---------|--------|--------|-------------------------------|
| add        | R      | 0       | 18     | 19     | 17      | 0      | 32     | add \$s1,\$s2,\$s3            |
| sub        | R      | 0       | 18     | 19     | 17      | 0      | 34     | sub \$s1,\$s2,\$s3            |
| lw         | I      | 35      | 18     | 17     | 100     |        |        | lw \$s1,100(\$s2)             |
| sw         | I      | 43      | 18     | 17     | 100     |        |        | sw \$s1,100(\$s2)             |
| and        | R      | 0       | 18     | 19     | 17      | 0      | 36     | and \$s1,\$s2,\$s3            |
| or         | R      | 0       | 18     | 19     | 17      | 0      | 37     | or \$s1,\$s2,\$s3             |
| nor        | R      | 0       | 18     | 19     | 17      | 0      | 39     | nor \$s1,\$s2,\$s3            |
| andi       | I      | 12      | 18     | 17     | 100     |        |        | andi \$s1,\$s2,100            |
| ori        | I      | 13      | 18     | 17     | 100     |        |        | ori \$s1,\$s2,100             |
| sll        | R      | 0       | 0      | 18     | 17      | 10     | 0      | sll \$s1,\$s2,10              |
| srl        | R      | 0       | 0      | 18     | 17      | 10     | 2      | srl \$s1,\$s2,10              |
| beq        | I      | 4       | 17     | 18     | 25      |        |        | beq \$s1,\$s2,100             |
| bne        | I      | 5       | 17     | 18     | 25      |        |        | bne \$s1,\$s2,100             |
| slt        | R      | 0       | 18     | 19     | 17      | 0      | 42     | slt \$s1,\$s2,\$s3            |
| j          | J      | 2       | 2500   |        |         |        |        | 10000 (see Section 2.9)       |
| jr         | R      | 0       | 31     | 0      | 0       | 0      | 8      | jr \$ra                       |
| jal        | J      | 3       | 2500   |        |         |        |        | jal 10000 (see Section 2.9)   |
| Field size |        | 6 bits  | 5 bits | 5 bits | 5 bits  | 5 bits | 6 bits | All MIPS instructions 32 bits |
| R-format   | R      | op      | rs     | rt     | rd      | shamt  | funct  | Arithmetic instruction format |
| I-format   | I      | op      | rs     | rt     | address |        |        | Data transfer, branch format  |

# Example of R Type

|           |       |             |
|-----------|-------|-------------|
| \$t0-\$t7 | 8-15  | temporaries |
| \$s0-\$s7 | 16-23 | saved       |

- add \$s1, \$s2, \$t1

| opcode | rs | rt | rd | shamt | funct |
|--------|----|----|----|-------|-------|
| 0      | 18 | 9  | 17 | xxx   | 32    |

# Practice on Your Own and convert them into binary



- srl \$s1, \$s2, 23

| opcode | rs  | rt | rd | shamt | funct |
|--------|-----|----|----|-------|-------|
| 0      | xxx | 18 | 17 | 23    | 02    |

- jr \$ra

| opcode | rs | rt  | rd  | shamt | funct |
|--------|----|-----|-----|-------|-------|
| 0      | 31 | xxx | xxx | xxx   | 08    |



# I Type Instruction



opcode → tells which I-type instruction (e.g. addi, lw, sw, beq, etc.)

rs → **base / first source register**

rt → can be **destination** OR source, depending on the instruction

immediate → a constant or address offset

# I Type Instruction

| op     | rs     | rt     | constant or address |
|--------|--------|--------|---------------------|
| 6 bits | 5 bits | 5 bits | 16 bits             |

Examples:

- `andi $s1, $s2, 223`

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 12     | 18 | 17 | 223    |

- `bne $s1, $s2, 100`

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 5      | 18 | 17 | 25     |

Divides  
by 4

# I Type Instruction



Examples:

\$s0 - \$s7= 16-23

- lw \$s1, 200(\$s2)

| opcode | Rs | Rt | C or A |
|--------|----|----|--------|
| 35     | 18 | 17 | 200    |

- sw \$s1, 200(\$s2)

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 43     | 18 | 17 | 200    |

# I Type Instruction

Examples:

\$s0 - \$s7 = 16-23

- lw \$s1, 200(\$s2)

| opcode | Rs | Rt | C or A |
|--------|----|----|--------|
| 35     | 18 | 17 | 200    |

- sw \$s1, 200(\$s2)

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 43     | 18 | 17 | 200    |

# I Type Instruction

Examples:

\$s0 - \$s7= 16-23

- lw **\$s1**, 200(\$s2)

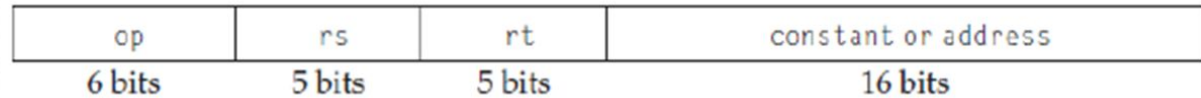
| opcode | Rs | <b>Rt</b> | C or A |
|--------|----|-----------|--------|
| 35     | 18 | 17        | 200    |

- sw \$s1, 200(\$s2)

| opcode | rs | <b>rt</b> | C or A |
|--------|----|-----------|--------|
| 43     | 18 | 17        | 200    |

# I Type Instruction

\$s0 - \$s7= 16-23



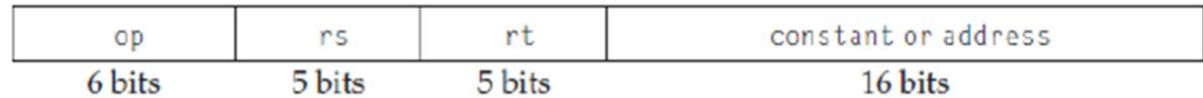
Examples:

- andi **\$s1**, \$s2, 223

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 12     | 18 | 17 | 223    |

# I Type Instruction

\$s0 - \$s7= 16-23



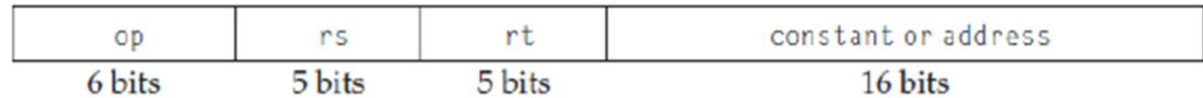
Examples:

- `andi $s1, $s2, 223`

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 12     | 18 | 17 | 223    |

# I Type Instruction

\$s0 - \$s7= 16-23



Examples:

- `andi $s1, $s2, 223`

| opcode | rs | rt | C or A |
|--------|----|----|--------|
| 12     | 18 | 17 | 223    |



# J Type MIPS

J-Type:

| opcode | address |
|--------|---------|
| 6 bits | 26 bits |

Examples:

- J 2000

| opcode | address          |
|--------|------------------|
| 02     | 500 Divides by 4 |

- Jal 10000

| opcode | address           |
|--------|-------------------|
| 03     | 2500 Divides by 4 |

- (d) Assume that processor X has 256 registers. The size of MIPS instruction is 32 bits and 6 bits are reserved for opcode. The structure for beq instruction is given in the table-1. **Find** out the maximum value of Y that can be written in the instruction below. [4]

| opcode | rs | rt | address |
|--------|----|----|---------|
|--------|----|----|---------|

Table 1: Structure of beq instruction

|   |                     |
|---|---------------------|
| 1 | beq \$s5, \$zero, Y |
|---|---------------------|

Processor X has 256 registers → needs 8 bits per register

Explanation:

Processor X has 256 registers. That means the registers are numbered:

0, 1, 2, 3, ..., 255

So we need to be able to represent any number from 0 to 255 in binary.

$\log_2(256)=8$ .

- (d) Assume that processor X has 256 registers. The size of MIPS instruction is 32 bits and 6 bits are reserved for opcode. The structure for beq instruction is given in the table-1. **Find** out the maximum value of Y that can be written in the instruction below. [4]

| opcode | rs | rt | address |
|--------|----|----|---------|
|--------|----|----|---------|

Table 1: Structure of beq instruction

|   |                     |
|---|---------------------|
| 1 | beq \$s5, \$zero, Y |
|---|---------------------|

Instruction size = 32 bits.

Opcode = 6 bits.

Each register is represented by 8 bits. Total 2 registers: rs, and rt.

Opcode- 6 bits, Rs- 8 bits, Rt- 8 bits, total 32 bits.

**32 – 6 – 8 – 8 = 10 bits left for address.**

- (d) Assume that processor X has 256 registers. The size of MIPS instruction is 32 bits and 6 bits are reserved for opcode. The structure for beq instruction is given in the table-1. **Find** out the maximum value of Y that can be written in the instruction below. [4]

| opcode | rs | rt | address |
|--------|----|----|---------|
|--------|----|----|---------|

Table 1: Structure of beq instruction

1

beq \$s5, \$zero, Y

address field = 10 bits → can represent values from -512 to +511 (signed 2's complement).

So maximum value of Y is 511.



Thank you!